

Q1 . Create a file that contains 1000 lines of random strings.

Code :

```
# Load necessary library
if (!require('stringi')) install.packages('stringi', dependencies=TRUE)
library(stringi)

# Define the number of lines and length of each string
num_lines <- 1000
string_length <- 10

# Generate random strings
random_strings <- stri_rand_strings(num_lines, string_length, pattern = "[A-Za-z]")

# Write to a file
write(random_strings, file = "random_strings.txt")

# Confirm file creation
cat("File 'random_strings.txt' created with", num_lines, "lines of random strings.\n")
```

O/P :

File 'random_strings.txt' created with 1000 lines of random strings.

Q2 Create a random dataset of 100 rows and 30 columns. All the values are defined between [1,200].

Code :

```
# Set the seed for reproducibility
set.seed(123)

# Generate the random dataset
data <- matrix(sample(1:200, 100*30, replace = TRUE), nrow = 100, ncol = 30)
data <- as.data.frame(data)

# Print the first few rows of the dataset
head(data)
```

O/P :

```

Console Terminal Background Jobs
R 4.3.1 ~ /
> head(data)
  V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18 V19 V20 V21 V22 V23
1 159 165 61 64 100 81 140 7 91 195 156 9 1 156 138 32 32 62 47 8 198 74 9
2 179 136 152 80 108 55 72 68 42 56 186 37 176 15 125 53 129 117 157 55 154 156 181
3 14 51 63 36 45 134 127 120 74 22 30 54 94 60 38 23 162 55 61 92 157 52 17
4 195 74 54 35 10 168 41 53 53 145 167 24 126 17 12 82 10 114 62 109 39 151 105
5 170 178 151 87 170 6 8 101 63 99 70 14 52 189 35 58 69 22 153 48 149 15 106
6 50 106 26 67 24 128 185 24 73 142 196 158 7 41 104 151 5 152 69 130 185 167 152
  V24 V25 V26 V27 V28 V29 V30
1 104 142 7 155 88 169 36
2 81 89 181 78 128 115 169
3 20 161 108 40 21 110 75
4 147 165 11 177 48 184 56
5 146 140 49 85 85 76 159
6 99 84 2 190 165 65 80
>

```

Perform the following operations:

- A) Replace all the values with NA in the dataset defined between [10, 60]. Print the count of number rows having missing values.

Code :

```

# Replace values between 10 and 60 with NA
data[data >= 10 & data <= 60] <- NA

# Count the number of rows with missing values
rows_with_na <- sum(apply(data, 1, function(row) any(is.na(row))))

print(paste("Number of rows with missing values:", rows_with_na))

```

O/P :

```
[1] "Number of rows with missing values: 100"
```

- b) Replace all the NA values with the average of the column value.

Code :

```

# Replace NA values with the column mean
for(i in 1:ncol(data)) {
  column_mean <- mean(data[[i]], na.rm = TRUE)
  data[is.na(data[[i]]), i] <- column_mean
}

# Print the first few rows to verify
head(data)

```

O/P :

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
1	159.0000	165.0000	61.0000	64.00	100.0000	81.0000	140.0000	7.0000	91.0000	195.0000
2	179.0000	136.0000	152.0000	80.00	108.0000	139.2405	72.0000	68.0000	118.3816	127.0278
3	118.5067	129.1831	63.0000	125.84	128.4648	134.0000	127.0000	120.0000	74.0000	127.0278
4	195.0000	74.0000	133.3067	125.84	128.4648	168.0000	110.0988	109.9296	118.3816	145.0000
5	170.0000	178.0000	151.0000	87.00	170.0000	6.0000	8.0000	101.0000	63.0000	99.0000
6	118.5067	106.0000	133.3067	67.00	128.4648	128.0000	185.0000	109.9296	73.0000	142.0000

	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20
1	156.0000	9.0000	1.0000	156.0000	138.0000	133.7083	120.1842	62.0000	119.3733	8.0000
2	186.0000	113.7286	176.0000	129.1644	125.0000	133.7083	129.0000	117.0000	157.0000	116.9467
3	136.8767	113.7286	94.0000	129.1644	112.5694	133.7083	162.0000	123.1562	61.0000	92.0000
4	167.0000	113.7286	126.0000	129.1644	112.5694	82.0000	120.1842	114.0000	62.0000	109.0000
5	70.0000	113.7286	131.1733	189.0000	112.5694	133.7083	69.0000	123.1562	153.0000	116.9467
6	196.0000	158.0000	7.0000	129.1644	104.0000	151.0000	5.0000	152.0000	69.0000	130.0000

	V21	V22	V23	V24	V25	V26	V27	V28	V29	V30
1	198.0000	74.0000	9.0000	104.0000	142	7.0000	155.0000	88.0000	169	122.5063
2	154.0000	156.0000	181.0000	81.0000	89	181.0000	78.0000	128.0000	115	169.0000
3	157.0000	110.3014	115.2405	119.2055	161	108.0000	127.1667	120.7051	110	75.0000
4	130.9474	151.0000	105.0000	147.0000	165	125.1642	177.0000	120.7051	184	122.5063
5	149.0000	110.3014	106.0000	146.0000	140	125.1642	85.0000	85.0000	76	159.0000
6	185.0000	167.0000	152.0000	99.0000	84	2.0000	190.0000	165.0000	65	80.0000

C) Find the Pearson correlation among all the columns and plot heat map. Also select those columns having correlation ≤ 0.7 .

Code :

```
# Calculate the Pearson correlation matrix
```

```
cor_matrix <- cor(data)
```

```
# Plot the heatmap
```

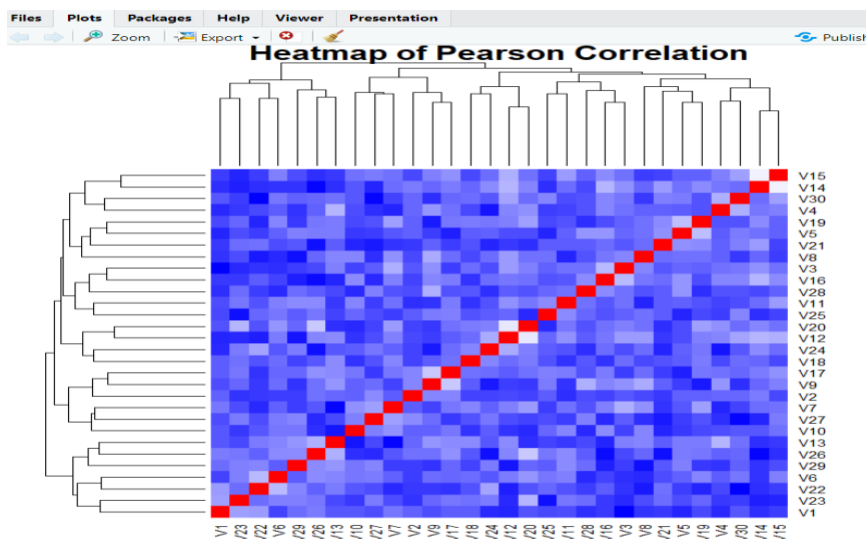
```
heatmap(cor_matrix, main = "Heatmap of Pearson Correlation", col =
colorRampPalette(c("blue", "white", "red"))(100), scale = "none")
```

```
# Select columns with correlation  $\leq 0.7$ 
```

```
selected_columns <- which(apply(cor_matrix, 1, function(x) all(x  $\leq$  0.7)), arr.ind = TRUE)
```

```
selected_data <- data[, selected_columns]
```

O/P :



D) Normalize all the values in the dataset between 0 and 10.

Code :

```
# Normalize values to the range 0 to 10

normalize <- function(x) {
  return ((x - min(x)) / (max(x) - min(x)) * 10)
}

data_normalized <- as.data.frame(lapply(data, normalize))

# Print the first few rows to verify

head(data_normalized)
```

O/P :

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
1	7.908163	8.241206	2.994924	3.144330	4.871795	3.865979	7.0769231	0.2525253	4.494949	10.000000
2	8.928571	6.783920	7.614213	3.969072	5.282051	6.868067	3.5897436	3.3333333	5.877858	6.478123
3	5.842177	6.441362	3.096447	6.331959	6.331528	6.597938	6.4102564	5.9595960	3.636364	6.478123
4	9.744898	3.668342	6.665313	6.331959	6.331528	8.350515	5.5435264	5.4509888	5.877858	7.409326
5	8.469388	8.894472	7.563452	4.329897	8.461538	0.000000	0.3076923	5.0000000	3.080808	5.025907
6	5.842177	5.276382	6.665313	3.298969	6.331528	6.288660	9.3846154	5.4509888	3.585859	7.253886
	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20
1	7.806122	0.3535354	0.0000000	7.743590	6.954315	5.234185	5.9787863	2.864583	5.886395	0.257732
2	9.336735	5.6428571	8.8832487	6.367404	6.294416	5.234185	6.4285714	5.729167	7.806122	5.873540
3	6.830445	5.6428571	4.7208122	6.367404	5.663424	5.234185	8.1122449	6.049805	2.908163	4.587629
4	8.367347	5.6428571	6.3451777	6.367404	5.663424	1.459854	5.9787863	5.572917	2.959184	5.463918
5	3.418367	5.6428571	6.6077834	9.435897	5.663424	5.234185	3.3673469	6.049805	7.602041	5.873540
6	9.846939	7.8787879	0.3045685	6.367404	5.228426	6.496350	0.1020408	7.552083	3.316327	6.546392
	V21	V22	V23	V24	V25	V26	V27	V28	V29	V30
1	9.897959	3.636364	0.4020101	5.202020	7.091837	0.255102	7.743590	4.285714	8.442211	6.231094
2	7.653061	7.777778	9.0452261	4.040404	4.387755	9.132653	3.794872	6.326531	5.728643	8.615385
3	7.806122	5.469766	5.7407290	5.969974	8.061224	5.408163	6.316239	5.954343	5.477387	3.794872
4	6.476907	7.525253	5.2261307	7.373737	8.265306	6.283887	8.871795	5.954343	9.195980	6.231094
5	7.397959	5.469766	5.2763819	7.323232	6.989796	6.283887	4.153846	4.132653	3.768844	8.102564
6	9.234694	8.333333	7.5879397	4.949495	4.132653	0.000000	9.538462	8.214286	3.216080	4.051282

>

E) Replace all the values in the dataset with 1 if value <=0.5 else with 0.

Code :

```
# Replace values with 1 if value <= 0.5 else with 0

data_binary <- as.data.frame(lapply(data_normalized, function(x) ifelse(x <= 0.5, 1, 0)))

# Print the first few rows to verify

head(data_binary)
```

O/P :

```
> head(scaled_mat)
  V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18 V19 V20 V21 V22 V23 V24 V25
1  0  0  0  0  0  0  0  1  0  0  0  1  1  0  0  0  0  0  0  1  0  0  1  0  0
2  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
3  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
4  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
5  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
6  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0
  V26 V27 V28 V29 V30
1    1    0    0    0    0
2    0    0    0    0    0
3    0    0    0    0    0
4    0    0    0    0    0
5    0    0    0    0    0
6    1    0    0    0    0
> |
```

3. Create a random dataset of 500 rows and 10 columns.

Columns 1 to 4 are defined between [-10, 10];

Columns 5 to 8 are defined between [10, 20];

Columns 9 to 10 are defined between [-100, 100]. Apply following clustering algorithms; determine the optimal number of clusters and plot distance metric graph using each algorithm.

Code :

```
# Set the seed for reproducibility
```

```
set.seed(123)
```

```
# Generate columns with specified ranges
```

```
col1_4 <- matrix(runif(500*4, min = -10, max = 10), ncol = 4)
```

```
col5_8 <- matrix(runif(500*4, min = 10, max = 20), ncol = 4)
```

```
col9_10 <- matrix(runif(500*2, min = -100, max = 100), ncol = 2)
```

```
# Combine all columns into a single data frame
```

```
data <- data.frame(col1_4, col5_8, col9_10)
```

```
# Print the first few rows of the dataset
```

```
head(data)
```

O/P:

```

      x1      x2      x3      x4      x1.1      x2.1      x3.1      x4.1      x1.2
1 -4.248450 -2.927878 -4.5275453  8.7605652 11.59674 18.53532 12.05827 12.41494 -39.10716
2  5.766103 -2.671171  1.8773387  9.7600661 11.44516 16.66007 19.42539 14.10778  66.56376
3 -1.820462 -4.257997 -6.7963037 -0.8736088 11.49180 17.33327 13.79324 18.11175  18.72950
4  7.660348 -8.400542  7.0686048 -5.3877011 15.14434 13.14740 16.26240 13.79141  61.43933
5  8.809346 -2.690915  6.9547832  3.9097854 14.92827 16.67718 11.83502 14.46950 -41.18984
6 -9.088870 -6.439724 -0.4422637  1.1326469 16.16343 14.63887 16.59208 15.70508 -71.78296
      x2.2
1 -78.991652
2 -19.498880
3  -8.864683
4  24.929049
5 -84.228944
6 -31.603455
> |

```

A) K-Mean clustering

Code :

```
# Load necessary library
```

```
library(ggplot2)
```

```
# Determine the optimal number of clusters using the Elbow method
```

```
wss <- (nrow(data) - 1) * sum(apply(data, 2, var))
```

```
for (i in 2:15) {
```

```
  wss[i] <- sum(kmeans(data, centers = i)$withinss)
```

```
}
```

```
# Plot the Elbow graph
```

```
elbow_plot <- data.frame(Clusters = 1:15, WSS = wss)
```

```
ggplot(elbow_plot, aes(x = Clusters, y = WSS)) +
```

```
  geom_line() +
```

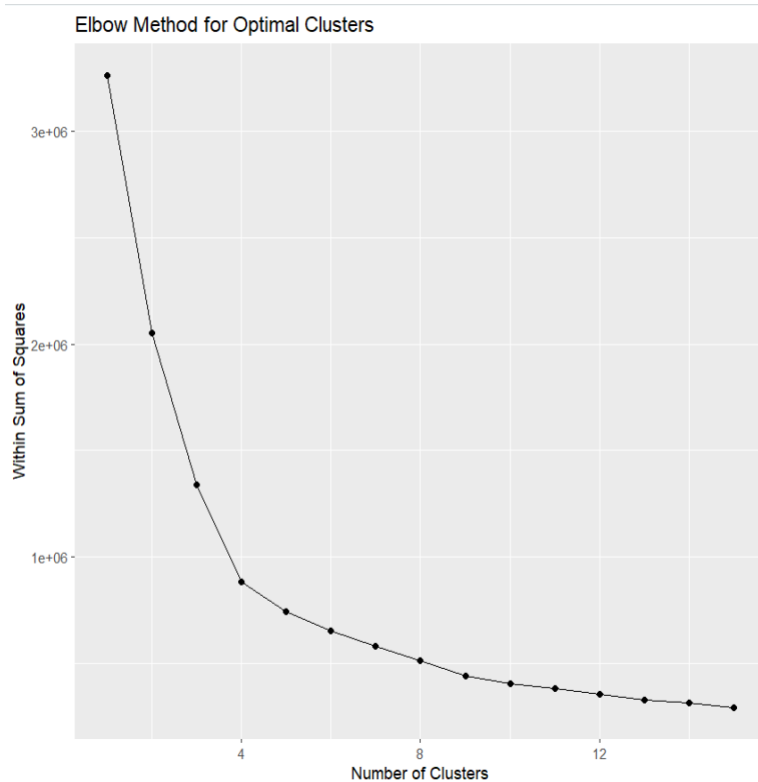
```
  geom_point() +
```

```
  ggtitle("Elbow Method for Optimal Clusters") +
```

```
  xlab("Number of Clusters") +
```

```
  ylab("Within Sum of Squares")
```

O/P :



(ii) Hierarchical clustering

Code :

```
# Compute the distance matrix
```

```
dist_matrix <- dist(data)
```

```
# Apply hierarchical clustering
```

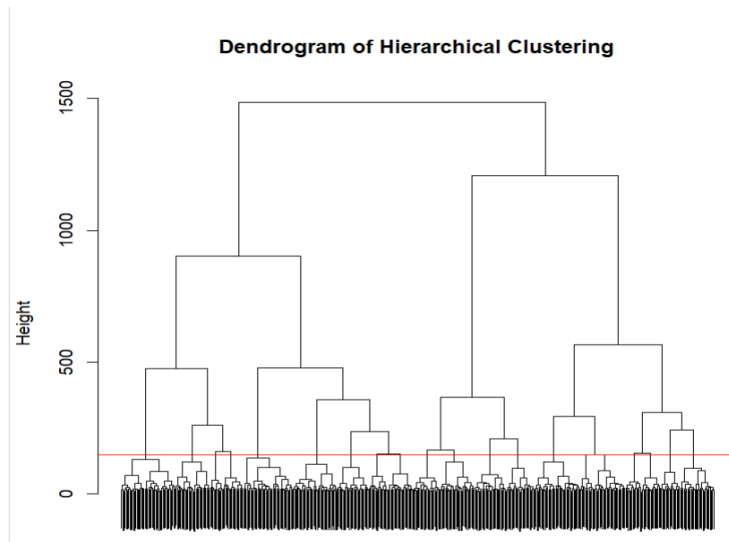
```
hclust_result <- hclust(dist_matrix, method = "ward.D2")
```

```
# Plot the dendrogram
```

```
plot(hclust_result, labels = FALSE, main = "Dendrogram of Hierarchical Clustering",  
xlab = "", sub = "")
```

```
abline(h = 150, col = "red") # Adding a horizontal line to cut the dendrogram at a chosen  
height
```

O/P :



Q3 .Create a random dataset of 600 rows and 15 columns. All the values are defined between [-100,100]. Perform the following operations:

Code :

```
# Set the seed for reproducibility
set.seed(123)

# Generate the random dataset with values between [-100, 100]
data <- matrix(runif(600*15, min = -100, max = 100), nrow = 600, ncol = 15)

data <- as.data.frame(data)

# Print the first few rows of the dataset
head(data)
```

O/P :

	V1	V2	V3	V4	V5	V6	V7	V8	V9
1	-42.48450	-52.55406	71.718281	-68.94717	46.46435	-58.83462	-80.272850	-15.8634393	-37.10002
2	57.66103	37.29807	77.476951	69.17020	21.94214	88.50781	24.689993	-90.2970679	-18.23485
3	-18.20462	-54.83632	-2.181700	-57.12391	-55.12557	-24.13524	-48.725448	30.1850933	-40.48216
4	76.60348	-36.30108	43.618362	33.97465	83.23420	25.24803	-4.146687	0.3795191	-75.97096
5	88.09346	-65.20324	-2.658885	23.55129	60.55220	-63.29952	-4.571299	57.2757050	-44.32561
6	-90.88870	60.28592	97.741772	-90.00004	-37.51828	31.84152	66.098283	9.4053117	-44.88818
	V10	V11	V12	V13	V14	V15			
1	-95.515656	-11.947796	-23.884034	-16.91590	-93.04311	48.494918			
2	5.673304	-20.521569	57.353631	-79.32863	-51.96111	-54.940917			
3	-16.307350	-25.690469	97.105047	76.90347	-13.77033	-62.930270			
4	94.959593	5.761715	1.673764	97.43211	36.69802	-37.865065			
5	-90.086400	-85.242918	61.553086	18.33091	65.97988	80.398650			
6	27.700245	43.369954	92.619152	80.92545	-68.37803	2.078657			

(i) Plot scatter graph between Column 5 and Column 6.

Code :


```
# Load necessary library

library(ggplot2)

# Plot scatter graph between Column 5 and Column 6

ggplot(data, aes(x = V5, y = V6)) +

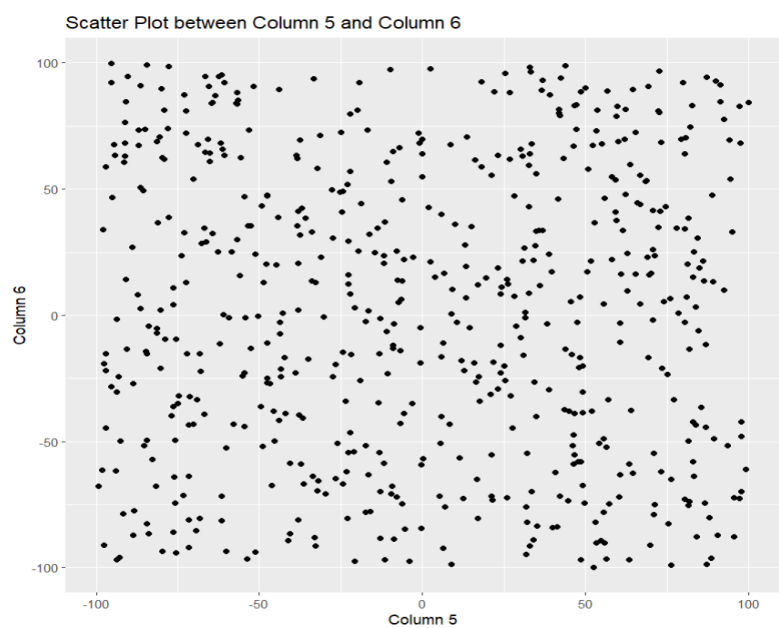
  geom_point() +

  ggtitle("Scatter Plot between Column 5 and Column 6") +

  xlab("Column 5") +

  ylab("Column 6")
```

O/P :



(i i) Plot histogram of each column in single graph.

Code :

```
# Reshape the data for plotting

library(reshape2)

data_melt <- melt(data)

# Plot histogram of each column in a single graph

ggplot(data_melt, aes(x = value)) +

  geom_histogram(bins = 30, fill = "blue", alpha = 0.7) +

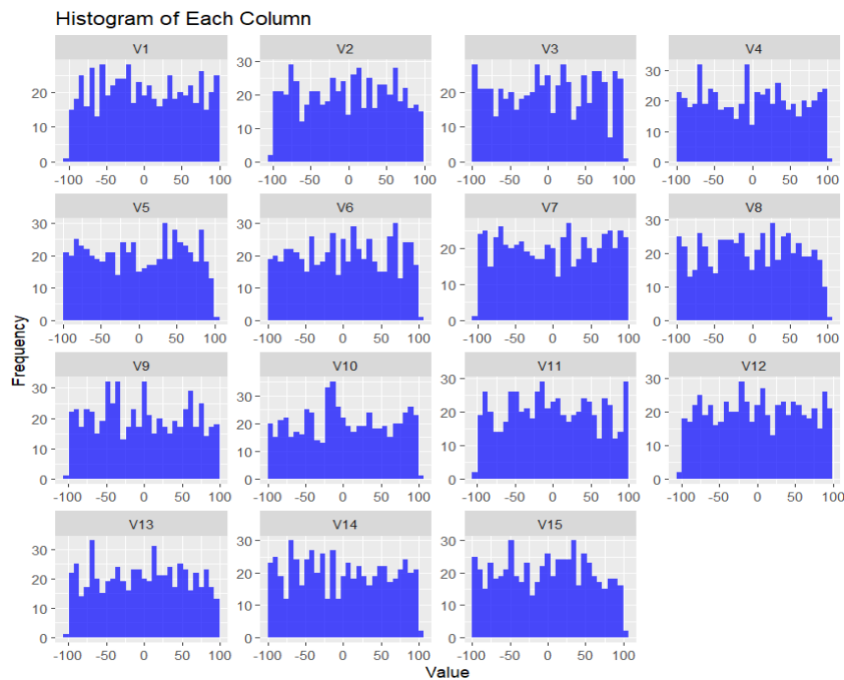
  facet_wrap(~ variable, scales = "free") +

  ggtitle("Histogram of Each Column") +

  xlab("Value") +

  ylab("Frequency")
```

O/P :

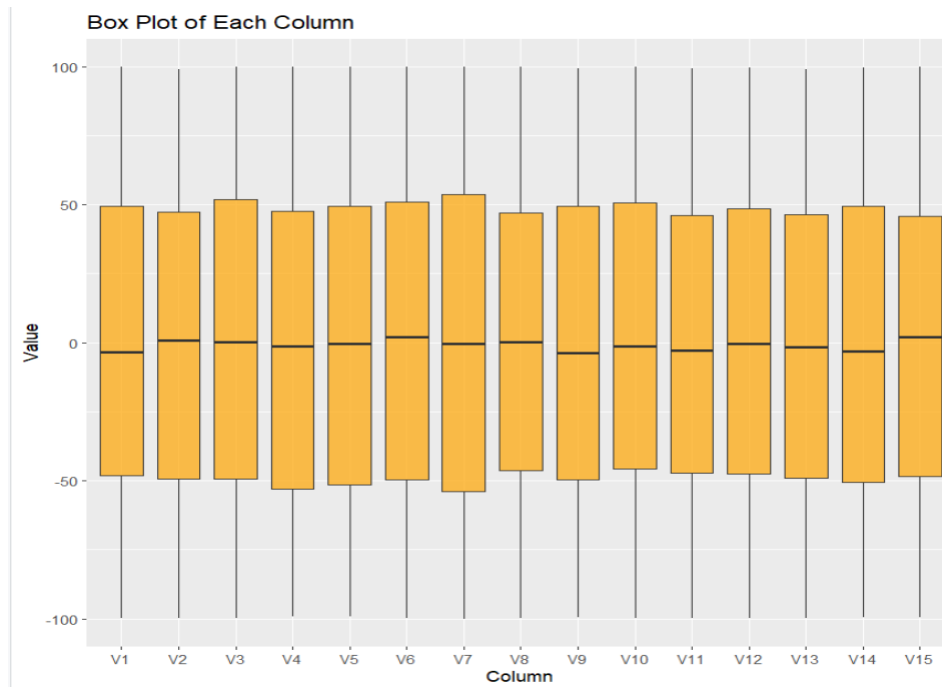


(i i i) Plot the Box plot of each column in single graph.

Code :

```
# Plot box plot of each column in a single graph
ggplot(data_melt, aes(x = variable, y = value)) +
  geom_boxplot(fill = "orange", alpha = 0.7) +
  ggtitle("Box Plot of Each Column") +
  xlab("Column") +
  ylab("Value")
```

O/P :



Q5. Create a random dataset of 500 rows and 5 columns: All the values are defined between [5,10].

Code :

Set the seed for reproducibility

```
set.seed(123)
```

Generate the random dataset with values between [5, 10]

```
data <- matrix(runif(500*5, min = 5, max = 10), nrow = 500, ncol = 5)
```

```
data <- as.data.frame(data)
```

Print the first few rows of the dataset

```
head(data)
```

O/P :

```

      v1      v2      v3      v4      v5
1 6.437888 6.768030 6.368114 9.690141 5.798370
2 8.941526 6.832207 7.969335 9.940017 5.722579
3 7.044885 6.435501 5.800924 7.281598 5.745902
4 9.415087 5.399865 9.267151 6.153075 7.572171
5 9.702336 6.827271 9.238696 8.477446 7.464137
6 5.227782 5.890069 7.389434 7.783162 8.081714
> |
```

(i) Perform t-Test on each column.

Code :

```
# Perform t-Test on each column
```

```
t_test_results <- apply(data, 2, function(column) t.test(column, mu = 7.5))
```

```
t_test_results
```

O/P :

```
$V1
      One Sample t-test

data:  column
t = -0.37079, df = 499, p-value = 0.711
alternative hypothesis: true mean is not equal to 7.5
95 percent confidence interval:
 7.351464 7.601373
sample estimates:
mean of x
 7.476418

$V2
      One Sample t-test

data:  column
t = -0.055988, df = 499, p-value = 0.9554
alternative hypothesis: true mean is not equal to 7.5
95 percent confidence interval:
 7.368610 7.624109
sample estimates:
mean of x
 7.49636

$V3
      One Sample t-test

data:  column
t = -0.4821, df = 499, p-value = 0.6299
alternative hypothesis: true mean is not equal to 7.5
95 percent confidence interval:
 7.345020 7.593909
sample estimates:
mean of x
 7.469464

$V4
      One Sample t-test

data:  column
t = 0.20319, df = 499, p-value = 0.8391
alternative hypothesis: true mean is not equal to 7.5
95 percent confidence interval:
 7.386422 7.639779
sample estimates:
mean of x
 7.513101

$V5
      One Sample t-test

data:  column
t = -0.69306, df = 499, p-value = 0.4886
alternative hypothesis: true mean is not equal to 7.5
95 percent confidence interval:
 7.324398 7.584020
sample estimates:
mean of x
 7.454209
```

(ii) Perform Wilcoxon Signed Rank Test on each column.

Code :

```
# Perform Wilcoxon Signed Rank Test on each column
```

```
wilcoxon_results <- apply(data, 2, function(column) wilcox.test(column, mu = 7.5))
```

```
wilcoxon_results
```

O/P :

```
$V1
```

```
      wilcoxon signed rank test with continuity correction
```

```
data: column
```

```
V = 61391, p-value = 0.7027
```

```
alternative hypothesis: true location is not equal to 7.5
```

```
$V2
```

```
      wilcoxon signed rank test with continuity correction
```

```
data: column
```

```
V = 62390, p-value = 0.9422
```

```
alternative hypothesis: true location is not equal to 7.5
```

```
$V3
```

```
      wilcoxon signed rank test with continuity correction
```

```
data: column
```

```
V = 61174, p-value = 0.6536
```

```
alternative hypothesis: true location is not equal to 7.5
```

```
$V4
```

```
      wilcoxon signed rank test with continuity correction
```

```
data: column
```

```
V = 63335, p-value = 0.8263
```

```
alternative hypothesis: true location is not equal to 7.5
```

```
$V5
```

```
      wilcoxon signed rank test with continuity correction
```

```
data: column
```

```
V = 60323, p-value = 0.4764
```

```
alternative hypothesis: true location is not equal to 7.5
```

(iii) Perform Two Sample t-Test and Wilcoxon Rank Sum Test on Column 3 and Column 4

Code :

```
# Two Sample t-Test on Column 3 and Column 4
```

```
t_test_two_sample <- t.test(data$V3, data$V4)
```

```
t_test_two_sample
```

```
# Wilcoxon Rank Sum Test on Column 3 and Column 4
```

```
wilcoxon_rank_sum_test <- wilcox.test(data$V3, data$V4)
```

```
wilcoxon_rank_sum_test
```

O/P :

```
Welch Two Sample t-test

data: data$V3 and data$V4
t = -0.48279, df = 997.68, p-value = 0.6293
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.2209985  0.1337259
sample estimates:
mean of x mean of y
 7.469464  7.513101
```

```
Wilcoxon rank sum test with continuity correction
```

```
data: data$V3 and data$V4
W = 122733, p-value = 0.6197
alternative hypothesis: true location shift is not equal to 0
```